

Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №4
по курсу «Вычислительная математика»
Вариант 19

Выполнил:

Якименко Владислав Игоревич, Р3213

Проверила:

Мальшева Татьяна Алексеевна

Содержание

1	Вычислительная реализация задачи	1
1.1	Постановка задачи	1
1.2	Графическое представление	1
1.3	Рабочие формулы метода	2
1.4	Таблица табулирования функции	4
1.5	Линейное приближение	4
1.6	Квадратичное приближение	5
1.7	Сравнение результатов	6
2	Программная реализация	6
2.1	Листинг программы	6
2.2	Результаты выполнения программы	8
2.3	Веб-интерфейс	9
2.4	Проверка программы	12
3	Вывод	12

1 Вычислительная реализация задачи

1.1 Постановка задачи

Задание:

1. Сформировать таблицу табулирования заданной функции на указанном интервале.
2. Построить линейное и квадратичное приближения по 11 точкам заданного интервала.
3. Найти среднеквадратические отклонения для каждой аппроксимирующей функции.
4. Выбрать наилучшее приближение.
5. Построить графики заданной функции, а также полученных линейного и квадратичного приближений.
6. Привести подробные вычисления.

Функция вычислительной части (вариант 19):

$$y = f(x) = \frac{5x}{x^4 + 19}, \quad x \in [0; 2], \quad h = 0.2.$$

1.2 Графическое представление

Построим график исходной функции $f(x)$, линейного приближения $\varphi_1(x)$ и квадратичного приближения $\varphi_2(x)$ на исследуемом интервале.

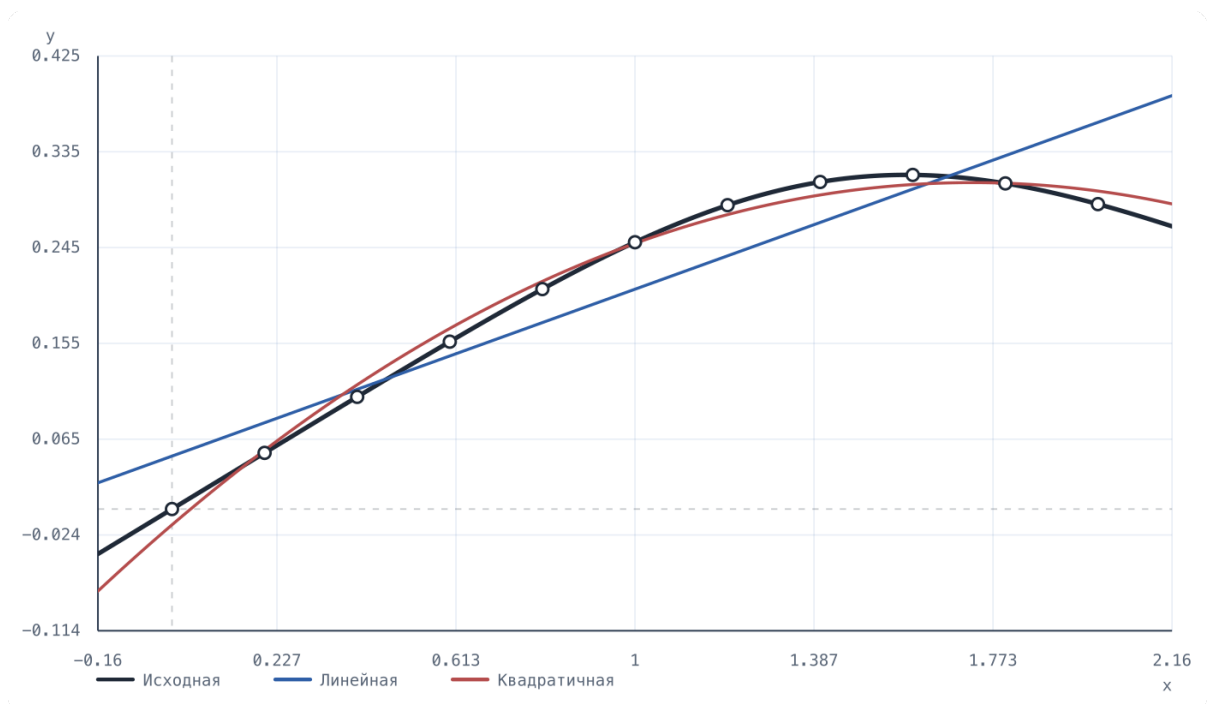


Рис. 1: Исходная функция и аппроксимирующие функции на отрезке $[0; 2]$

По графику видно, что линейная функция описывает только общий рост табличных значений, а квадратичная функция лучше учитывает максимум функции на правой части интервала.

1.3 Рабочие формулы метода

Мерой отклонения аппроксимирующей функции $\varphi(x)$ от табличной функции является сумма квадратов отклонений:

$$S = \sum_{i=1}^n (\varphi(x_i) - y_i)^2 \rightarrow \min.$$

Среднеквадратическое отклонение:

$$\delta = \sqrt{\frac{S}{n}}.$$

Для полинома степени m

$$\varphi(x) = a_0 + a_1x + a_2x^2 + \dots + a_mx^m$$

коэффициенты находятся из нормальной системы:

$$\sum_{j=0}^m a_j \sum_{i=1}^n x_i^{j+k} = \sum_{i=1}^n x_i^k y_i, \quad k = 0, 1, \dots, m.$$

Для линейной зависимости дополнительно вычисляется коэффициент корреляции Пирсона:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}.$$

Коэффициент детерминации:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \varphi(x_i))^2}{\sum_{i=1}^n (y_i - \bar{y})^2}.$$

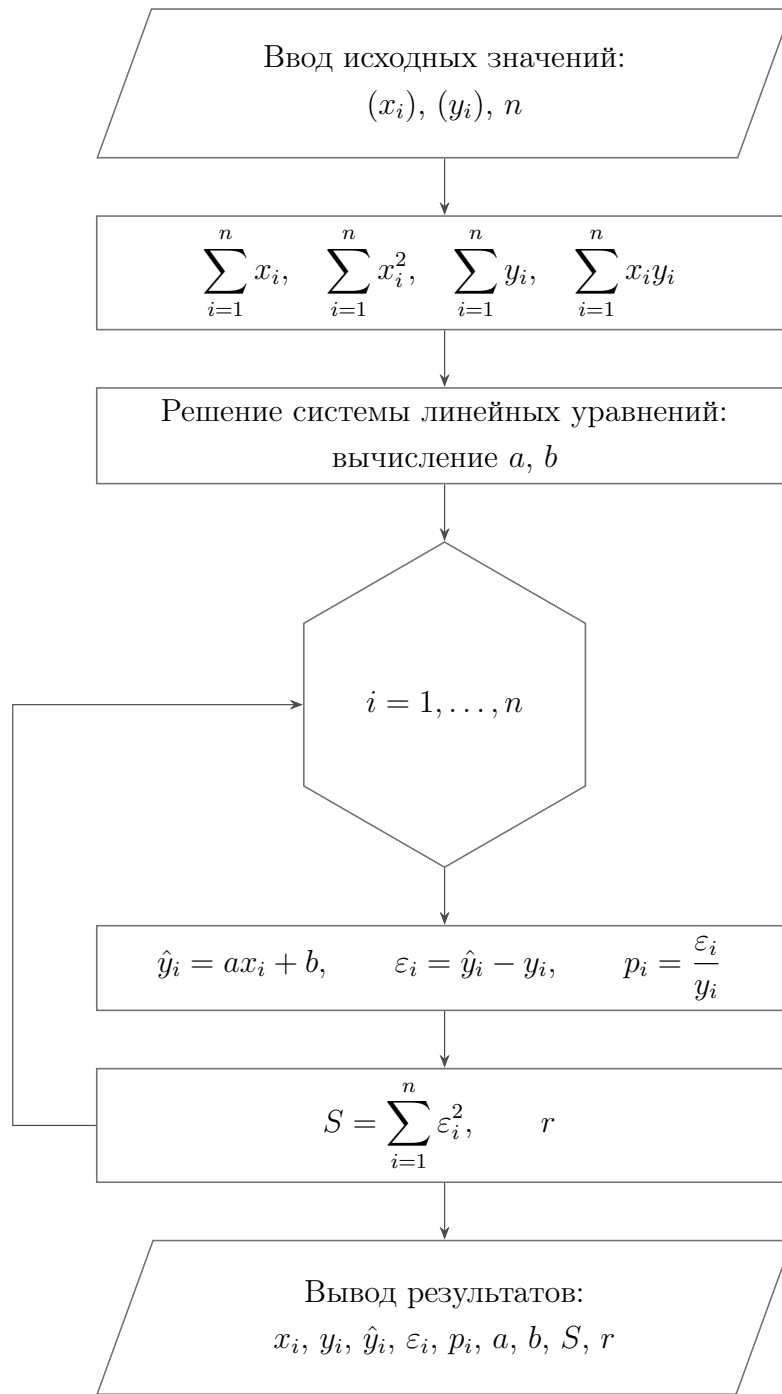


Рис. 2: Блок-схема алгоритма линейной аппроксимации методом наименьших квадратов

1.4 Таблица табулирования функции

Таблица 1: Точки табулирования функции $f(x) = \frac{5x}{x^4 + 19}$

i	x_i	y_i	i	x_i	y_i
1	0.0	0.000000	7	1.2	0.284716
2	0.2	0.052627	8	1.4	0.306458
3	0.4	0.105122	9	1.6	0.313067
4	0.6	0.156825	10	1.8	0.305110
5	0.8	0.206084	11	2.0	0.285714
6	1.0	0.250000			

Вычисленные суммы для нормальных уравнений:

Таблица 2: Суммы для метода наименьших квадратов

Величина	Значение	Величина	Значение
n	11	$\sum y_i$	2.265723
$\sum x_i$	11.000000	$\sum x_i y_i$	2.953771
$\sum x_i^2$	15.400000	$\sum x_i^2 y_i$	4.400790
$\sum x_i^3$	24.200000	$\sum x_i^3 y_i$	7.076887
$\sum x_i^4$	40.532800	$\sum x_i^5$	70.664000
$\sum x_i^6$	126.617920		

1.5 Линейное приближение

Ищем функцию вида

$$\varphi_1(x) = a_1 x + a_0.$$

Нормальная система:

$$\begin{cases} 11a_0 + 11a_1 = 2.265723, \\ 11a_0 + 15.4a_1 = 2.953771. \end{cases}$$

Решение системы:

$$a_0 = 0.049600, \quad a_1 = 0.156374.$$

Итак,

$$\boxed{\varphi_1(x) = 0.156374x + 0.049600}.$$

Таблица 3: Значения линейного приближения

i	x_i	y_i	$\varphi_1(x_i)$	ε_i
1	0.0	0.000000	0.049600	0.049600
2	0.2	0.052627	0.080875	0.028248
3	0.4	0.105122	0.112150	0.007029
4	0.6	0.156825	0.143425	-0.013400
5	0.8	0.206084	0.174700	-0.031384
6	1.0	0.250000	0.205975	-0.044025
7	1.2	0.284716	0.237250	-0.047467
8	1.4	0.306458	0.268525	-0.037934
9	1.6	0.313067	0.299800	-0.013268
10	1.8	0.305110	0.331074	0.025965
11	2.0	0.285714	0.362349	0.076635

$$S_1 = 0.016825, \quad \delta_1 = \sqrt{\frac{0.016825}{11}} = \boxed{0.039}.$$

Коэффициент корреляции Пирсона для линейной зависимости:

$$r = 0.929929.$$

1.6 Квадратичное приближение

Ищем функцию вида

$$\varphi_2(x) = a_2x^2 + a_1x + a_0.$$

Нормальная система:

$$\begin{cases} 11a_0 + 11a_1 + 15.4a_2 = 2.265723, \\ 11a_0 + 15.4a_1 + 24.2a_2 = 2.953771, \\ 15.4a_0 + 24.2a_1 + 40.5328a_2 = 4.400790. \end{cases}$$

Решение системы:

$$a_0 = -0.014787, \quad a_1 = 0.370999, \quad a_2 = -0.107312.$$

Итак,

$$\boxed{\varphi_2(x) = -0.107312x^2 + 0.370999x - 0.014787}.$$

Таблица 4: Значения квадратичного приближения

i	x_i	y_i	$\varphi_2(x_i)$	ε_i
1	0.0	0.000000	-0.014787	-0.014787
2	0.2	0.052627	0.055120	0.002493
3	0.4	0.105122	0.116443	0.011321
4	0.6	0.156825	0.169180	0.012355
5	0.8	0.206084	0.213332	0.007249
6	1.0	0.250000	0.248900	-0.001100
7	1.2	0.284716	0.275882	-0.008834
8	1.4	0.306458	0.294280	-0.012179
9	1.6	0.313067	0.304092	-0.008975
10	1.8	0.305110	0.305320	0.000210
11	2.0	0.285714	0.297962	0.012248

$$S_2 = 0.001016, \quad \delta_2 = \sqrt{\frac{0.001016}{11}} = \boxed{0.010}.$$

1.7 Сравнение результатов

Таблица 5: Сравнение линейного и квадратичного приближений

Приближение	S	δ
Линейное	0.016825	0.039
Квадратичное	0.001016	0.010

Наилучшим среди двух приближений является квадратичное, так как $\delta_2 < \delta_1$.

2 Программная реализация

Проект написан на JavaScript. Консольная часть запускается через Node.js, для визуализации дополнительно сделан web-интерфейс в браузере. Программа принимает таблицу из 8–12 точек, строит все требуемые аппроксимирующие функции, вычисляет меру отклонения S , среднеквадратическое отклонение δ , коэффициент детерминации R^2 , коэффициент корреляции Пирсона для линейной модели и выбирает лучшую функцию по минимальному значению δ .

2.1 Листинг программы

Построение нормальной системы для полиномиальной аппроксимации:


```

1 export function leastSquaresPolynomial(points, degree) {
2   const size = degree + 1;
3   const matrix = Array.from({ length: size }, () => Array(size).fill(0));
4   const vector = Array(size).fill(0);
5
6   for (let row = 0; row < size; row += 1) {
7     for (let column = 0; column < size; column += 1) {
8       matrix[row][column] = sum(points.map((point) =>
9         point.x ** (row + column)));
10    }
11    vector[row] = sum(points.map((point) =>
12      point.y * point.x ** row));
13  }
14
15  return gaussianElimination(matrix, vector);
16 }

```

Расчёт статистик аппроксимации:

```

1 function calculateStats(points, evaluate) {
2   const values = points.map((point) => {
3     const approximation = evaluate(point.x);
4     const residual = approximation - point.y;
5     return { x: point.x, y: point.y, approximation, residual };
6   });
7
8   const s = sum(values.map((row) => row.residual ** 2));
9   const sigma = Math.sqrt(s / points.length);
10  const yMean = mean(points.map((point) => point.y));
11  const totalScatter = sum(points.map((point) =>
12    (point.y - yMean) ** 2));
13  const r2 = 1 - s / totalScatter;
14
15  return { values, s, sigma, r2 };
16 }

```

Линеаризация неполиномиальных функций:

```

1 function fitExponential(points) {
2   requireDomain(points, (point) => point.y > 0);
3   const transformed = points.map((point) =>
4     ({ x: point.x, y: Math.log(point.y) }));
5   const [lnA, b] = leastSquaresPolynomial(transformed, 1);
6   const a = Math.exp(lnA);
7   return createModel("exponential", points, { a, b },
8     (x) => a * Math.exp(b * x));

```

```

9 }
10
11 function fitPower(points) {
12   requireDomain(points, (point) => point.x > 0 && point.y > 0);
13   const transformed = points.map((point) =>
14     ({ x: Math.log(point.x), y: Math.log(point.y) }));
15   const [lnA, b] = leastSquaresPolynomial(transformed, 1);
16   const a = Math.exp(lnA);
17   return createModel("power", points, { a, b }, (x) => a * x ** b);
18 }

```

2.2 Результаты выполнения программы

Для исходного набора варианта 19 программа получила следующие результаты:

Таблица 6: Результаты программы для варианта 19

Модель	S	δ	R^2
Линейная	0.01682546	0.03910993	0.86476730
Полиномиальная 2-й степени	0.00101641	0.00961256	0.99183068
Полиномиальная 3-й степени	0.00002656	0.00155375	0.99978656
Экспоненциальная	не применима	—	—
Логарифмическая	не применима	—	—
Степенная	не применима	—	—

Экспоненциальная, логарифмическая и степенная модели неприменимы к исходной таблице варианта 19, поскольку в ней есть точка $(0; 0)$, а линеаризация этих моделей требует положительных x и/или y .

Лучшее применимое приближение:

$$\varphi(x) = -0.050036x^3 + 0.042797x^2 + 0.256515x - 0.000376.$$

Для положительного тестового набора, где применимы все шесть моделей, получены результаты:

Таблица 7: Результаты программы для положительного тестового набора

Модель	S	δ	R^2
Линейная	0.00047389	0.00725633	0.99984697
Полиномиальная 2-й степени	0.00047385	0.00725605	0.99984698
Полиномиальная 3-й степени	0.00042496	0.00687155	0.99986277
Экспоненциальная	0.04569543	0.07125497	0.98524409
Логарифмическая	0.09150904	0.10083487	0.97045003
Степенная	0.01081343	0.03466255	0.99650814

Для некорректного набора из семи точек программа завершилась с диагностикой:

Ошибка: Таблица должна содержать от 8 до 12 точек.

2.3 Веб-интерфейс

Для лабораторной работы реализован web-интерфейс, позволяющий редактировать таблицу точек, загружать данные из файла, запускать расчёт, просматривать таблицу отклонений и график. График можно масштабировать кнопками или колесом мыши, а также перемещать перетаскиванием.

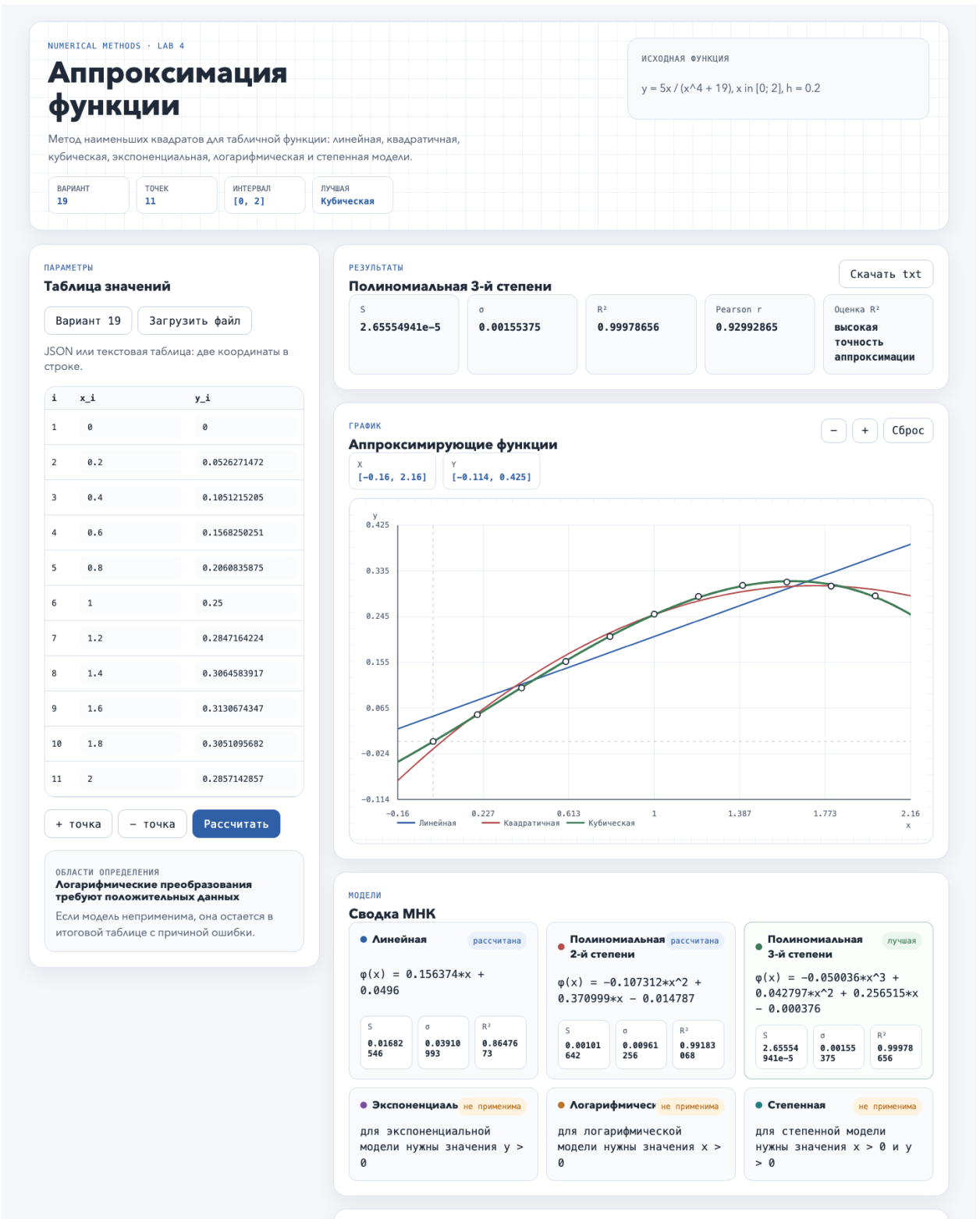


Рис. 3: Веб-интерфейс: таблица исходных данных, метрики и интерактивный график

NUMERICAL METHODS · LAB 4

Аппроксимация функции

Метод наименьших квадратов для табличной функции: линейная, квадратичная, кубическая, экспоненциальная, логарифмическая и степенная модели.

ВАРИАНТ
19

ТОЧЕК
11

ИНТЕРВАЛ
[0, 2]

ЛУЧШАЯ
Кубическая

ИСХОДНАЯ ФУНКЦИЯ
 $y = 5x / (x^4 + 19), x \in [0; 2], h = 0.2$

ПАРАМЕТРЫ

Таблица значений

Вариант 19

Загрузить файл

JSON или текстовая таблица: две координаты в строке

i	x_i	y_i
1	0	0
2	0.2	0.0526271472
3	0.4	0.1051215205
4	0.6	0.1568250251
5	0.8	0.2060835875
6	1	0.25
7	1.2	0.2847164224
8	1.4	0.3064583917
9	1.6	0.3130674347
10	1.8	0.3051095682
11	2	0.2857142857

+ точка

- точка

Рассчитать

ОБЛАСТИ ОПРЕДЕЛЕНИЯ
Логарифмические преобразования требуют положительных данных

Рис. 4: Веб-интерфейс в узком окне: адаптивная форма ввода данных

2.4 Проверка программы

Таблица 8: Наборы данных для проверки

Набор	Содержание проверки	Результат
Вариант 19	11 точек, $x \in [0; 2]$, присутствует $(0; 0)$	расчёт выполнен
Положительный набор	9 точек, $x > 0, y > 0$	рассчитаны все 6 моделей
Некорректный набор	7 точек вместо 8–12	выведена ошибка

Автоматические тесты проекта проверяют табулирование варианта 19, восстановление точного квадратичного полинома методом МНК, обработку моделей с ограничениями области определения и чтение текстовых данных с десятичной запятой.

3 Вывод

- Для вычислительной части сформирована таблица из 11 точек функции $f(x) = \frac{5x}{x^4+19}$ на отрезке $[0; 2]$ с шагом $h = 0.2$.
- Линейное приближение имеет среднеквадратическое отклонение $\delta_1 = 0.039$, квадратичное — $\delta_2 = 0.010$; следовательно, квадратичное приближение лучше.
- В программной части исследованы все требуемые функции: линейная, полиномиальная 2-й степени, полиномиальная 3-й степени, экспоненциальная, логарифмическая и степенная.
- На исходных данных варианта 19 лучшим применимым приближением стала полиномиальная функция 3-й степени с $\delta = 0.00155375$.
- В общем случае модель выбирается по минимальному δ и наибольшему R^2 , но также учитывается вид зависимости: линейная модель подходит для почти прямолинейных данных, полином 2-й степени — при одном изгибе, а полином 3-й степени имеет смысл выбирать, если он заметно уменьшает ошибку.
- Экспоненциальная, логарифмическая и степенная модели полезны, когда данные соответствуют их форме роста и выполняются ограничения области определения ($x > 0$ и/или $y > 0$).
- Экспоненциальная, логарифмическая и степенная функции для исходного набора неприменимы из-за точки $(0; 0)$; на положительном тестовом наборе программа корректно рассчитывает все шесть моделей.

- Программа обрабатывает некорректные данные и выводит диагностическое сообщение, если таблица содержит меньше 8 или больше 12 точек.